Métodos numéricos para computación gráfica

Interacción, Gráficos y Multimedia

Máster en Ingeneiría Informática

Contenido

1. Método de Monte Carlo

2. Curvas y superficies
   - Curvas de Bézier
   - Curvas B-Spline
   - Superficies de Bézier

Contenido

1. Método de Monte Carlo

2. Curvas y superficies
   - Curvas de Bézier
   - Curvas B-Spline
   - Superficies de Bézier

Método de Monte Carlo. Introducción

Método de Monte Carlo. Idea

El término método de Monte Carlo abarca un amplio rango de técnicas de resolución de problemas que se basan en el uso de números aleatorios y probabilidad estadística.

- El método de Monte Carlo fue bautizado así por su analogía con los juegos de ruleta de los casinos, siendo el más célebre el de Monte Carlo, inaugurado en 1861.

Método de Monte Carlo. Historia

Historia del Método de Monte Carlo

- 1733: Problema de la agua de Buffon
- 1812: Laplace sugiere usar el experimento de la aguja de Buffon para estimar el valor de π
- 1946: Se construyó ENIAC (Electronic Numerical Integrator and Computer), considerado el primer ordenador de propósito general. Utilizado por el Laboratorio de Investigación Balística del Ejército de los Estados Unidos
- 1947: John von Neuman y Stanislaw Ulam proponen una simulación en ordenador para resolver el problema de la difusión de neutrones en una material fisionable
- 1949: Metropolis y Ulam publican sus resultados en la revista *Journal of the American Statistical Association*

Método de Monte Carlo. Aproximar π

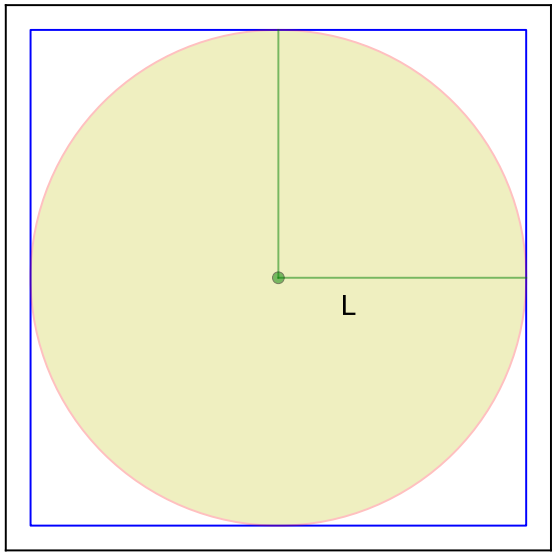
- Consideramos un cuadrado de lado $2L$ que contenga un círculo inscrito (el radio es igual a L).
- Relación entre las áreas:

$$\frac{\text{Área Círculo}}{\text{Área Cuadrado}} = \frac{\pi L^2}{4L^2} = \frac{\pi}{4}$$

- Valor de π:

$$\pi = 4. * \frac{\text{Área Círculo}}{\text{Área Cuadrado}}$$

- Esta relación de áreas es la misma en el cuadrante superior derecho

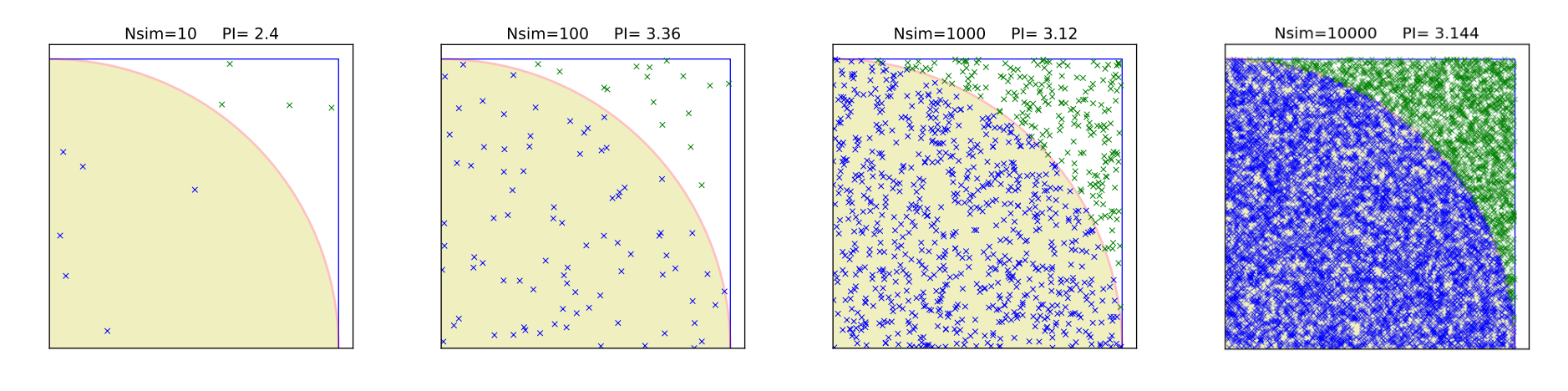


Método de Monte Carlo. Aproximar π

Idea. Cálcular el valor de π

Lanzar dardos al azar sobre el cuadrante superior (las coordenadas x, y de cada punto están comprendidas entre 0 y 1), de tal forma que contabilizaremos los puntos que caen dentro del semicírculo, $N_{aciertos}$.

$$\pi = 4 * \frac{A_{circulo}}{A_{cuadrado}} \approx 4 * \frac{N_{aciertos}}{N_{total}}$$



Método de Monte Carlo. Aproximar π

Listing 1: Código en Python

```
import random

nsim=1000
ptosIn=0
for i in range(0,nsim):
    x=random.random()
    y=random.random()
    if x**2+y**2<=1:ptosIn+=1

vpi=4.*float(ptosIn)/float(nsim)
```

Observación

Estamos aproximando la integral:

$$\int\int_R dxdy = \pi, \quad R = \{(x,y) \in \mathbb{R}^2 / x^2 + y^2 \leq 1\}$$

Método de Monte Carlo. Ideas previas

- Se llama **variable aleatoria (v.a.)**, X, a toda aplicación que asocia a cada elemento del espacio muestral (Ω) de un experimento, un número real. (Una v.a. se corresponde con una variable estadística cuyos valores se obtienen de mediciones en algún tipo de experimento aleatorio).
- f es una **función de densidad** de una v.a. X (continua) si:
  - $f(x) \geq 0$ en todos los valores donde está definida
  - $\int_{-\infty}^{\infty} f(x)dx = 1$ (el área bajo la curva es uno)

  La probabilidad se determina como: $P(a < X < b) = \int_a^b f(x)dx$
- La **función de distribución**, F, asociada a la v.a. X se define como:, $F(x) = Prob(X \leq x) = \int_{-\infty}^{x} f(x)dx$
- La **esperanza matemática** (también llamada **valor esperado** o **media**), de una v. a. X, $E(X)$ o $\mu = E(X)$, es el número que formaliza la idea de valor medio de un fenómeno aleatorio. Se define como:
  - Si X es continua: $E(X) = \int_{-\infty}^{\infty} xf(x)dx$

Método de Monte Carlo. Ideas previas

- Si X es una v.a. con una densidad f y $g : \mathbb{R} \to \mathbb{R}$ una función, entonces el valor esperado de la v.a. $g(X)$ es:

$$E[g(X)] = \int_{-\infty}^{\infty} g(x)f(x)dx$$

- **Ley Fuerte de los Grandes Números**: Si X_1, X_2, ... es una sucesión de v.a. independientes e idénticamente distribuidas (i.i.d.), todas con media μ entonces

$$\lim_{N\to\infty} \frac{X_1 + X_2 + \ldots + X_N}{N} = \mu$$

- Si $\{X_N\}_{N=1,2,\ldots}$ es una sucesión de v.a. i.i.d $\Longrightarrow$ $\{g(X_N)\}_{N=1,2,\ldots}$ es una sucesión de v.a. i.i.d.

Método de Monte Carlo. Planteamiento del problema

Como consecuencia de la Ley Fuerte de los Grandes Números: Si consideramos una sucesión de v.a. i.i.d. $\{X_N\}_{N=1,2,\ldots}$:

$$E[g(X)] = \lim_{N\to\infty} \frac{g(X_1) + g(X_2) + \ldots + g(X_N)}{N}$$

Método de Monte Carlo

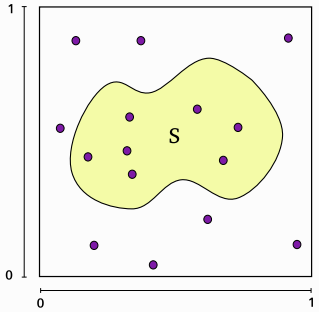
1. Generamos una muestra i.i.d. de un vector de v.a. X: $X_1, X_2, \ldots, X_N$
2. Estimamos la media poblacional $E[g(X))]$ por la media muestral

$$E[g(X)] \approx \frac{1}{N}\sum_{i=1}^{N} g(X_i)$$

- Método robusto
- Método cuya convergencia es lenta: $O\left(\frac{1}{\sqrt{N}}\right)$

Método de Monte Carlo. Estimación de un área

- ¿Cómo podemos aproximar el área A de una región S?



Método de Aceptación-Rechazo

- Si X_1, $X_2 \sim U(0,1)$ (uniformes en $[0,1]$), el área A de S es:

$$A = E[g(X_1, X_2)]$$

donde

$$g(x_1, x_2) = \mathbb{1}_{(x_1,x_2)\in S} = \begin{cases} 1 \text{ si } (x_1, x_2) \in S \\ 0 \text{ en otro caso} \end{cases}$$

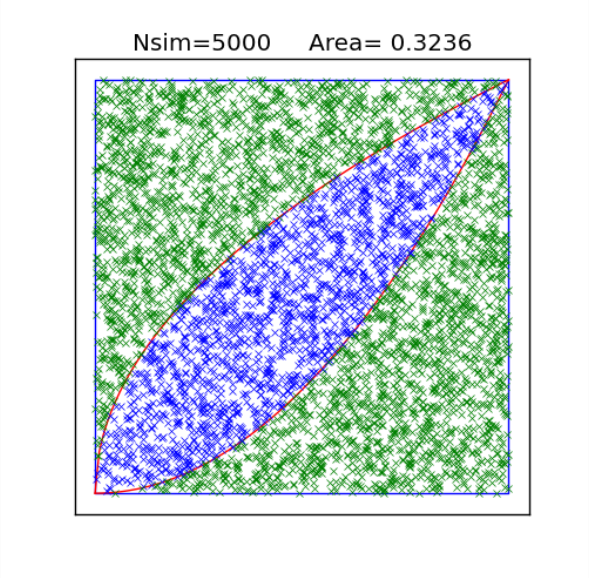
- Generamos una muestra i.i.d. de tamaño N de un par de v.a. independientes $\boldsymbol{X} = (X_1, X_2)$ con X_1, $X_2 \sim U(0,1)$: $\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_N$ con $\boldsymbol{X}_k = (X_{1k}, X_{2k})$
- Estimación del área mediante el método de Monte Carlo:

$$A = E[g(X_1, X_2)] \approx \frac{1}{N} \sum_{k=1}^{N} g(X_{1k}, X_{2k})$$

Método de Monte Carlo. Estimación de un área

Ej: Calcular el área de la región $R = (x, y) \in \mathbb{R}^2 / x^2 \leq y \leq \sqrt{(x)}\}$ (Área= 1/3)

```
import random, math

ptosIn=0
nsim=1000
for i in range(0,nsim):
    x=random.random() #U(0,1)
    y=random.random() #U(0,1)

    if x**2<=y<=math.sqrt(x):ptosIn+=1

area=float(ptosIn)/float(nsim)
```



Método de Monte Carlo. Integración

- Calcular la integral definida $I = \int_0^1 g(x)dx$
- Considerando una v.a. $X \sim U(0,1)$, podemos representar la integral como una esperanza:

$$\int_0^1 g(x)dx = E[g(X)]$$

Estimación de la integral mediante el método de Monte Carlo:

- Generar una muestra i.i.d $X_1, X_2, \ldots, X_N \sim U(0,1)$
- Estimar la integral por la media muestral:

$$I \approx \frac{1}{N}\sum_{i=1}^{N} g(X_i)$$

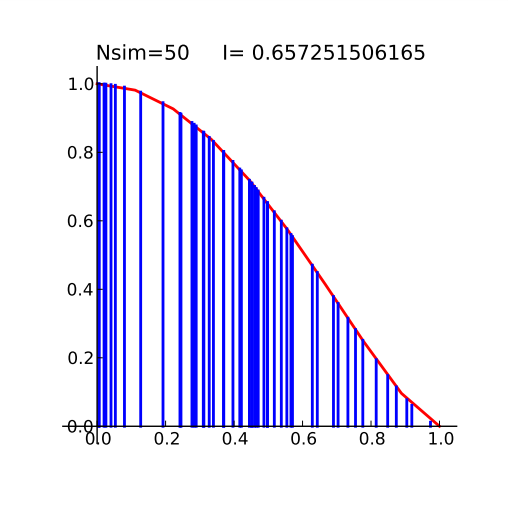
- Aproximación de la integral definida de g de a a b:

$$\int_a^b g(x)dx = (b-a)\int_0^1 g(a+(b-a)x)dx \approx \frac{b-a}{N}\sum_{i=1}^{N} g(a+(b-a)X_i) \text{ con } X_i \sim U(0,1)$$

Método de Monte Carlo. Integración

Ej: Aproximar $I = \int_0^1 g(x)dx$, con $g(x) = (1-x^2)^{3/2}$ $\left(I = \frac{3\pi}{16} \approx 0{,}5890486225480862\right)$

```
import random,math
g=lambda x:(1.-x*x)**(3./2.)
nsim=50
integral=0.
for i in range(0,nsim):
    x=random.random() #U(0,1)
    gx=g(x)
    integral+=gx

integral/=float(nsim)
```



N	Aprox
10	0,779043385957
100	0,599232416817
1000	0,566597762729
10000	0,593608909947
100000	0,588554450716

Método de Monte Carlo. Integrales

Comparación de la velocidad de convergencia

- La velocidad de convergencia del método de Monte Carlo es $O\left(\frac{1}{\sqrt{n}}\right)$
- La velocidad de convergencia de un método de integración numérica de una función unidimensional empleando sumas de Riemann es $O\left(\frac{1}{n}\right)$
- En el caso unidimensional Monte Carlo no compensa. Sin embargo:
  - El orden de convergencia del método de Monte Carlo es independiente de la dimensión
  - El orden de convergencia de técnicas de integración numérica, como sumas de Riemann deteriora al aumentar la dimensión
- El método de Monte Carlo puede ser una buena elección para integrales multidimensionales

Método de Monte Carlo. Integrales

- Integral múltiple:

$$I = \int_0^1 \int_0^1 \dots \int_0^1 g(x_1, \dots, x_k) dx_1 dx_2 \dots dx_k$$

- Se verifica que:

$$I = E[g(X_1, \dots X_k)]$$

  con $X_1, \dots, X_k$ independientes y uniformes en $(0, 1)$
- Aproximación mediante el método de Monte Carlo:

Generamos N conjuntos de k números aleatorios uniformes en $(0, 1)$

$$\begin{array}{ccc} X_{1,1}, & \dots, & X_{k,1} \\ X_{1,2}, & \dots, & X_{k,2} \\ & \vdots & \\ X_{1,N}, & \dots, & X_{k,N} \end{array}$$

Estimamos I mediante

$$I \approx \frac{1}{N} \sum_{i=1}^{N} g\left(X_{1,i}, \dots, X_{1,i}\right)$$

Método de Monte Carlo. Integrales

Ej: $I = \int_0^1 \dots \int_0^1 2^{-7} \left(\sum_{i=1}^{8} x_i \right)^2 dx_1\, dx_2 \dots dx_n \quad \left(I = \frac{25}{192} \approx 0{,}130208 \right)$

```
import random
nsim=100
integral=0.
for i in range(0,nsim):
    x=[]
    for k in range(0,8): x.append(random.random()) #U(0,1)
    gx=sum(x)**2
    integral+=gx

integral*=2.**(-7)
integral/=float(nsim)
```

N	Aprox
10	0,111233864468
100	0,127757496532
1000	0,131928226113
10000	0,130114828202
100000	0,130295932876

Método de Monte Carlo. Aplicaciones

- Ray Tracing
- Criptografía
- Finanzas (por ej.: pronóstico del índice de la bolsa de valores, valoración de cartera de valores)
- Radioterapia contra el cáncer
- Densidad y flujo de tráfico
- Diseño de reactores nucleares
- Evolución estelar
- Ecología
- Econometría
- Física de materiales
- Programas de ordenador
- Prospecciones en explotaciones petroleras
- Seguros

Método de Monte Carlo. Principales componentes

Los principales elementos que componen el Método de Monte Carlo son:

- Generador de números aleatorios
- Regla simple
- Función de Distribución de probabilidad
- Error de estimación

Método de Monte Carlo. Números aleatorios

- Paradoja filosófica:
  - Necesitamos generar números aleatorioes por ordendor
  - No podemos: un ordenador es una máquina determinista
- En un ordenador podemos generar **números pseudo-aleatorios**: se construyen de forma determinista, pero parecen aleatorios y se puede usar como tales.
- Los métodos de generación de números aleatorios se utilizan para obtener números aleatorios entre 0 y 1
- Se pueden generar números pseudo-aleatorios para casi cualquier distribución a partir de números pseudo-aleatorios uniformes en $(0,1)$

Método de Monte Carlo. Números aleatorios

- Los algoritmos para generar números pseudo-aleatorios en el intervalo $(0,1)$ deben tener buenas propiedades:
  - Uniformidadad: Los números deben distribuirse uniformemente en el intervalo $(0,1)$
  - Independencia: los números deben ser independientes (si $x_1, x_2, \ldots, x_n$ son números pseudo-aleatorios en el intervalo $(0,1)$ estos no deben contener ninguna información apreciable sobre x_{n+1})
  - Periodo largo: deben permitirnos generar un gran número de ellos sin que se comiencen a repetir
  - Rápido y eficiente: generador rápido y que emplee poca memoria
  - Reproducible: que se pueda reproducir la misma secuencia de números
  - Portable: el funcionamiento del generador es el mismo independientemente del sistema operativo, lenguaje y hardware.

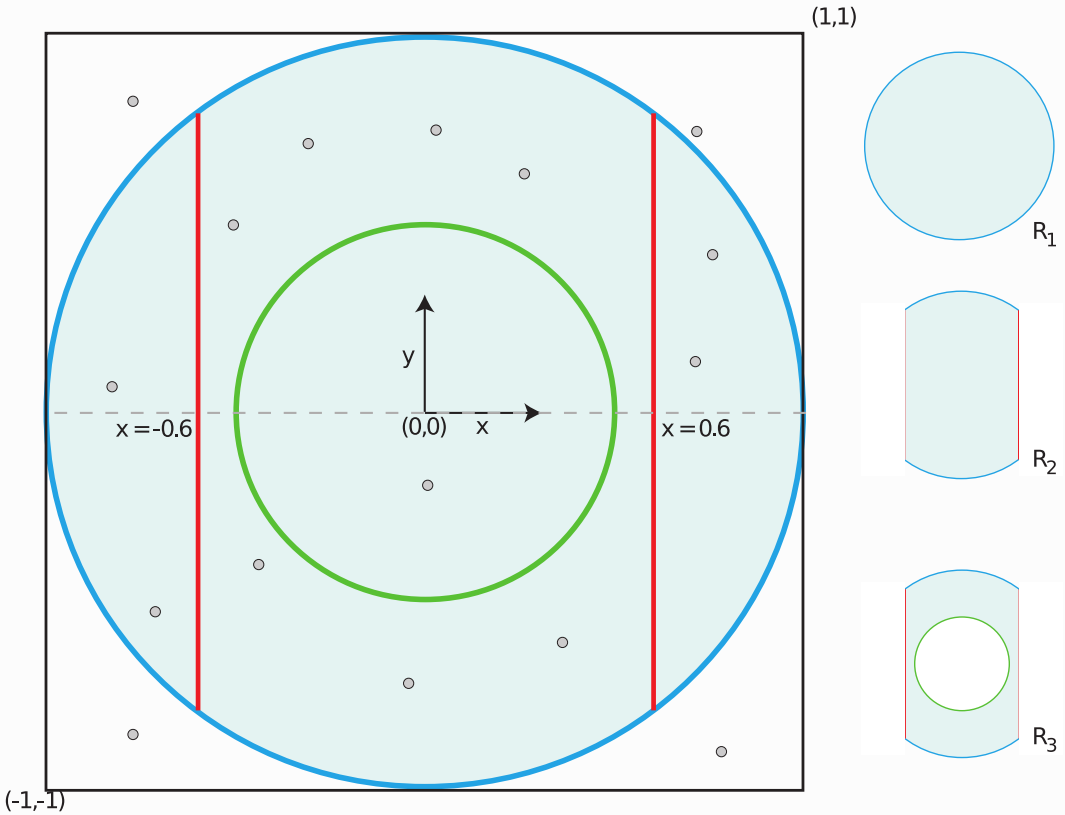
Método de Monte Carlo. Números aleatorios

Elección de un buen generador de números aleatorios:

- Atendiendo al problema, para algunas aplicaciones se prefiere la velocidad, mientras que para otras son más importantes la robusted y la fiabilidad.
  - Para una simulación con Monte Carlo las propiedades distributivas de los generadores aleatorios son de suma importancia
  - En criptografía la aleatoriedad es crucial
- Dos generadores que en general ofrecen un buen rendimiento son:
  - **Mersenne-twister**: generador (basado en un método matricial de congruencias lineales) muy rápido (es de los más rápidos debido a su implementación en binario) que generar números pseudo-aleatorios de gran calidad, y que tiene un periodo muy largo ($2^{19937} - 1$). Generador por defecto en Python, Matlab, GNU.
  - **L'Ecuyer**: generador lineal recursivo múltiple que tienen propiedades estadísticas excelentes, es simple, tienen un largo periodo (2^{191}), soportan múltiples flujos y es relativamente rápido.

Método de Monte Carlo. Ejercicio

- Aproximar el área de las regiones R_1, R_2, R_3



Contenido

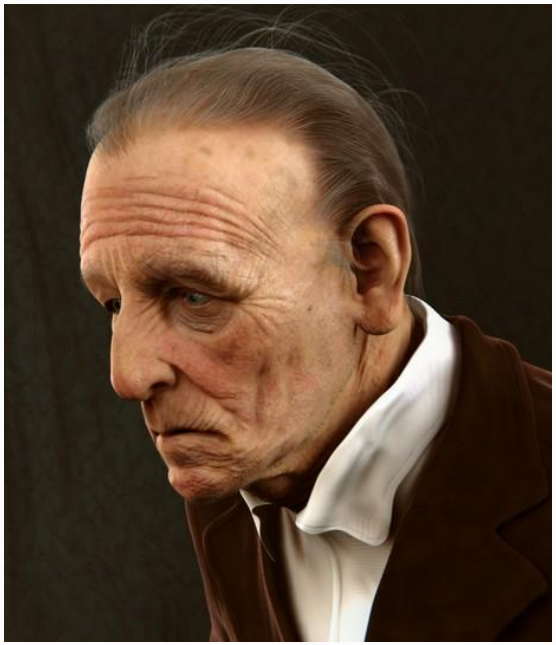
1 Método de Monte Carlo

2 Curvas y superficies
- Curvas de Bézier
- Curvas B-Spline
- Superficies de Bézier

Curvas y Superficies. Motivación

- No podemos representar objetos, que reflejen la realidad, sólo con líneas, polígonos y objetos simples (esferas)
- En los cuerpos de los seres vivos hay curvas
- Los objetos que se nos rodean tienen curvas

[1]



[1] Figuras de Blender.org

La tetera de Utah

- ¿Cuántas veces habéis visto una tetera renderizada en películas, video juevos o en salvapantallas

- Hasta 1975 renderizar una tetera suponía un auténtico reto
- En 1975 Martin Newell desarrolló un modelo matemático para renderizar una tetera

Curvas y Superficies. Motivación

¿Cómo modelar y/o representar objetos reales?

- Problema:
  No hay modelo matemático del objetos
- Solución:
  Realizar una aproximación por trozos de planos, esperas y otras formas "simples" de modelar
- Se intenta que el modelo sea lo más cercano posible al objeto real

Curvas y Superficies. Motivación

- Las curvas y superficies permiten el modelado de formas suaves:
  - Caricaturas
  - Curvas que rigen el movimiento en animación
- La suavidad puede ser garantizada analíticamente
- Representación analítica
- Teoría de suavidad de curvas y superficies es bien conocida (cálculo y geometría diferencial)

Curvas y Superficies. Representación

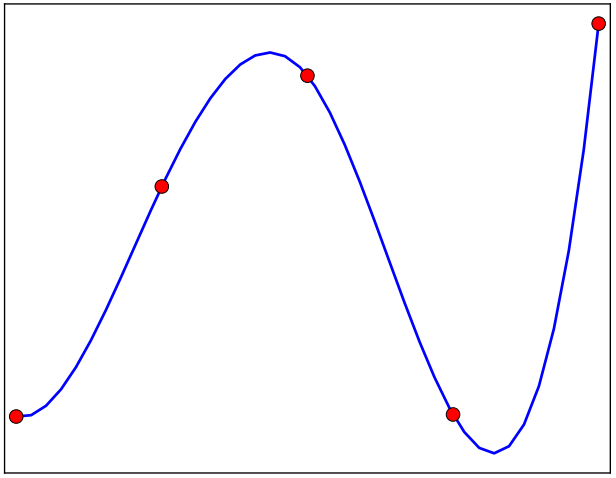
Representación

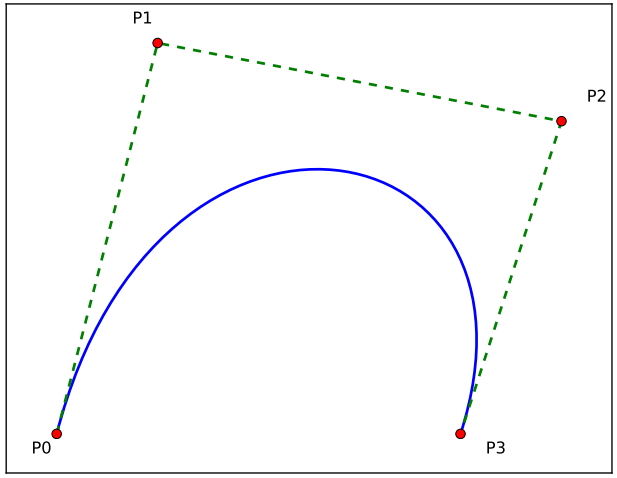
- Forma explícita: $y = f(x), z = f(x, y)$. Ej: $y = sqrt(x^2 + 1)$
- Forma implícita: $f(x, y) = 0, f(x, y, z) = 0$. Ej.: $x^2 + y^2 = 1$
- Paramétrica:
  - Curva 2D: $x = x(t)$, $y = y(t)$. Ej: $x = \cos(t)$, $y = \sin(t)$
  - Curva 3D: $x = x(t)$, $y = y(t)$, $z = z(t)$
  - Superficie paramétrica: $x = x(u, v)$, $y = y(u, v)$, $z = z(u, v)$
- La representación **paramétrica** es más flexible

Curvas

Técnicas constructivas:

- **Interpolación**: se genera una curva pasando por puntos dados
- **Ajuste/Aproximación**: la curva se define a través de puntos exteriores que controlan su forma

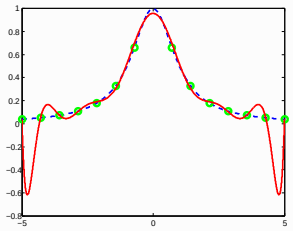




Curvas

Clases de curvas:

- Por sencillez y coste computacional, en representación gráfica se utilizan **polinomios o cocientes de polinomios** (funciones racionales).
- **Polinomios de grado n**
  - Interpolación de Lagrange
  - Interpolación de Hermite
  - Problemas:
    - Grado alto para obtener flexibilidad
    - Coste computacional elevado si el grado es alto
    - Se producen oscilaciones
    - No hay control en la forma de la curva
    - Los polinomios más usados son los cúbicos (grado 3)



- **Curvas polinómicas o racionales a trozos**
  - Curvas de Bezier: función polinómica a trozos de clase $\mathcal{C}^1$ (derivada 1ª continua)
  - Curvas Spline: función polinómica a trozos de clase $\mathcal{C}^2$ (derivada 2ª continua)
  - NURBS (Non-Uniform Rational B-Spline: curvas racionales compuestas

Curvas de Bézier

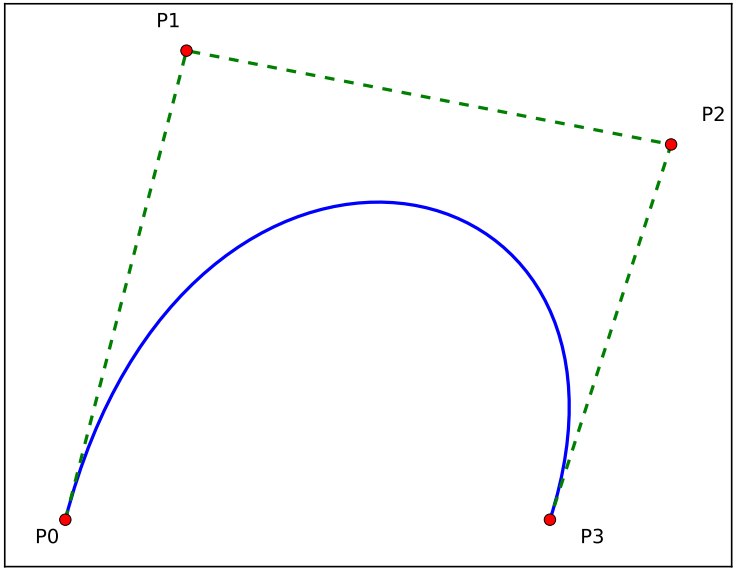
Historia

- Históricamente se trazaban planos con junquillo (en inglés, spline) en al industria naval, automovilística, aeronáutica, ...
- Uno de los primeros métodos modernos de modelado de curvas fue desarrollado de forma simultánea por los ingenieros Paul de Casteljau en Citroën y Pierre Bezier en Renault
- En 1966 Bézier publicó publicó sus investigaciones en 1966, lo que no hizo de Casteljau
- El diseño geométrico asistido por ordenador tiene su origen esencialmente en la industria del automóvil

Curvas de Bézier

Idea intuitiva

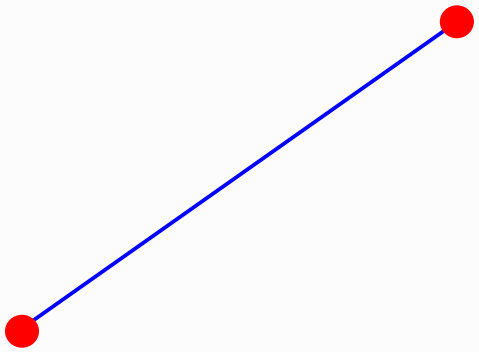
- Se define por una serie de puntos de control
- La curva siempre empieza en el primer punto, y termina en el último
- Los puntos intermedios "atraen" hacia si a la curva



Curvas de Bezier. Lineales (Grado 1)

- Definidas por dos puntos de control: P_0 y P_1
- Son rectas
- Podemos recorrer la curva con un parámetro $t \in [0, 1]$ que recorre la recta de P_0 a P_1

$$B(t) = (1 - t)P_0 + tP_1$$

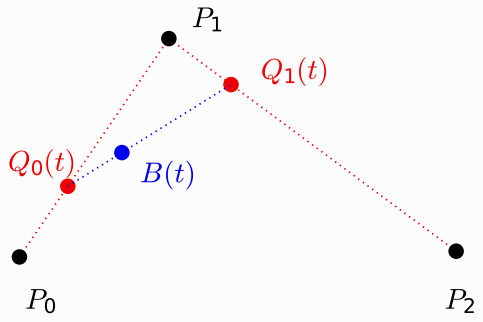


Curvas de Bezier. Cuadráticas (Grado 2)

- Se usan tres puntos de control: P_0, P_1, P_3
- Se construyen dos curvas lineales de Bezier entre los puntos $P_0 - P_1$ y $P_1 - P_2$
- Se cosntruye una tercera curva lineal de Bezier entre las dos anteriores
- Variando $t \in [0, 1]$ a lo largo de la tercera recta, se construye la curva

$$
\begin{aligned}
Q_0(t) &= (1-t)P_0 + tP_1 \\
Q_1(t) &= (1-t)P_1 + tP_2
\end{aligned}
$$

$$
\begin{aligned}
B(t) &= (1-t)Q_0(t) + tQ_1(t) \\
&= (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2
\end{aligned}
$$



Curvas de Bézier. Cuadráticas (Grado 2) (cont.)

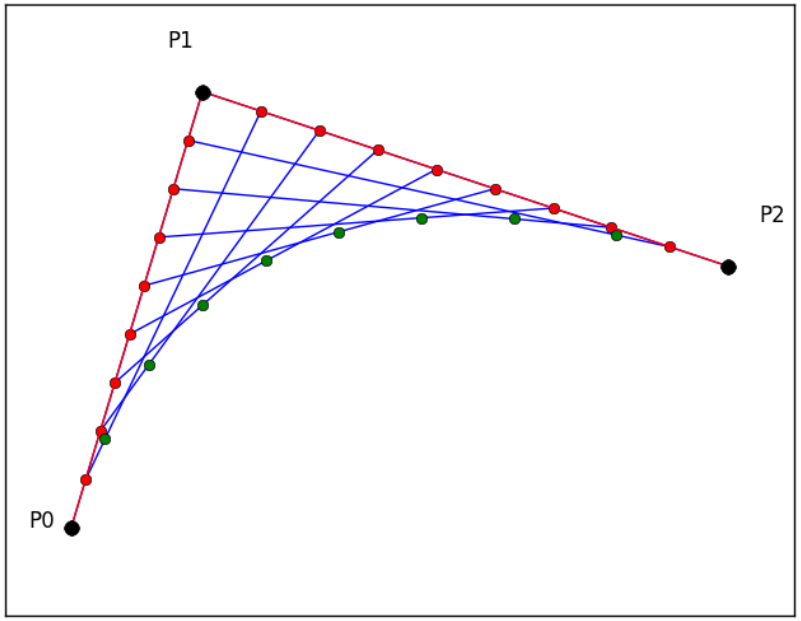
- Pasa por los extremos:

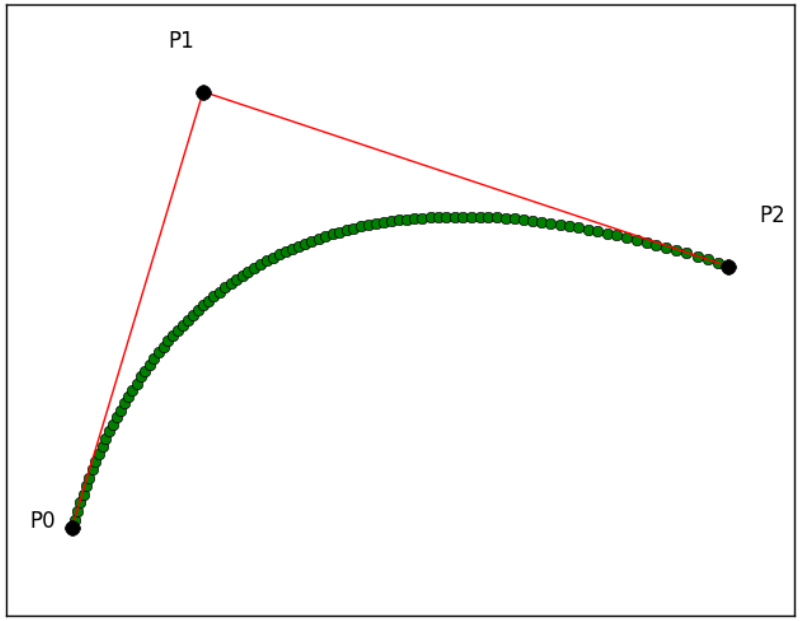
$$B(0) = P_0, \quad B(1) = P_2$$

- Tangentes en los extremos:

$$B'(0) = 2(P_1 - P_0), \quad B'(1) = 2(P_2 - P_1)$$

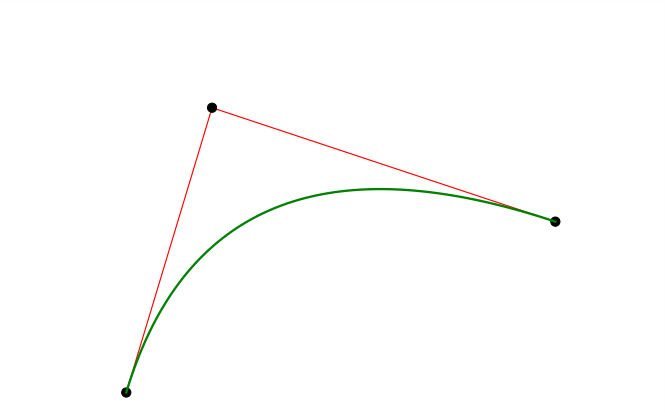
Las tangentes en los extremos pasan por P_1





Curvas de Bézier. Cuadráticas (Grado 2) (cont.)

```
import matplotlib.pyplot as plt
import numpy as np

x=np.array([0.,0.2,1. ])
y=np.array([0.5,1.,0.8])
t=np.linspace(0.,1.,100)

bx=(1-t)**2*x[0]+2*t*(1-t)*x[1]+t**2*x[2]
by=(1-t)**2*y[0]+2*t*(1-t)*y[1]+t**2*y[2]

fig1=plt.figure(1)
plt.plot(x,y,'r-',x,y,'ko',ms=8)
plt.plot(bx,by,'g-',lw=2)

plt.xticks([])
plt.yticks([])
plt.axis([x.min()-0.1, x.max()+0.1, y.min()-0.1, y.max()+0.1])
plt.show()
```

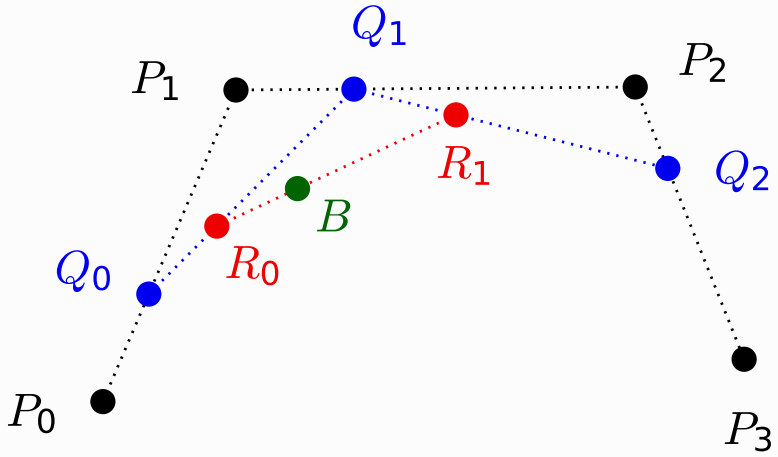


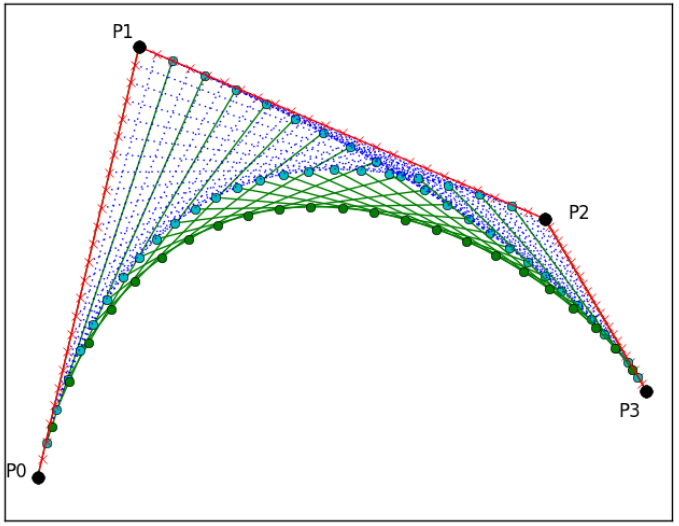
Curvas de Bézier. Cúbicas (Grado 3)

- Curva de Bézier de grado 3:

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P3$$

También se obtiene interpolando en tres capas con los puntos de control:

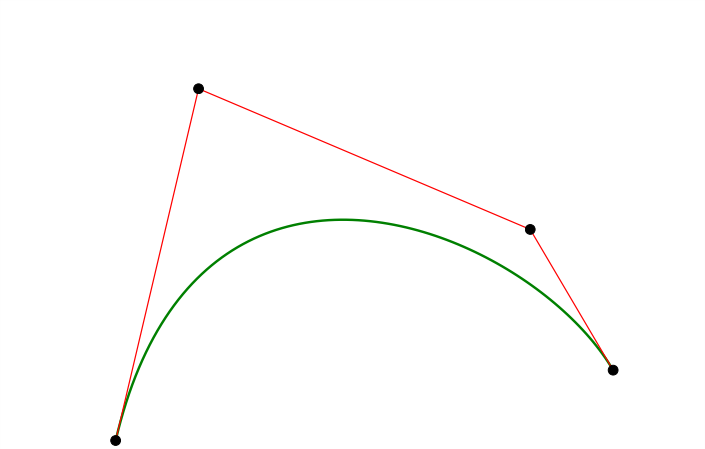




- Obs: trabajaremos siempre en el intervalo $t \in [0,1]$
  Si es necesario trabajar en el intervalo $[a,b]$, es suficiente escribir $B\left(\frac{t-a}{b-a}\right)$

Curvas de Bézier. Cúbicas (Grado 3) (cont.)

```
import matplotlib.pyplot as plt
import numpy as np

fig=plt.figure(1)
t=np.linspace(0.,1.,105)

Bx=(1-t)**3*x[0]+3*t*(1-t)**2*x[1]+3.*t**2*(1-t)*x[2]+t**3*x[3]
By=(1-t)**3*y[0]+3*t*(1-t)**2*y[1]+3.*t**2*(1-t)*y[2]+t**3*y[3]

plt.plot(Bx,By,'g-',lw=2)
plt.plot(x,y,'r-',x,y,'ko',ms=8)

plt.xticks([])
plt.yticks([])
plt.axis('off')
plt.axis([min(x)-0.065,max(x)+.05,min(y)-0.05,max(y)+0.05])
plt.show()
```

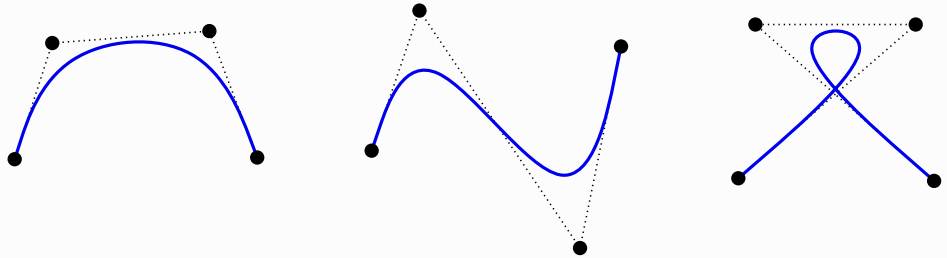


Curvas de Bézier. Grado n

- En general, si tenemos $n+1$ puntos de control, la curva de Bézier de grado n:

$$B(t) = \sum_{i=0}^{n} B_{i,n}(t) P_i = \sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} P_i; \qquad \binom{n}{i} = \frac{n!}{i!(n-i)!}$$

  donde P_i son los puntos de control, y $B_{i,n}$ son los polinomios de Berstein
- **cuando aumenta n**, se consigue mejorar la aproximación, pero con dos inconvenientes:
  1. los cálculos se complican
  2. los cambios locales (en un solo punto de control) afectan a toda la curva



Curvas de Bézier. Propiedades

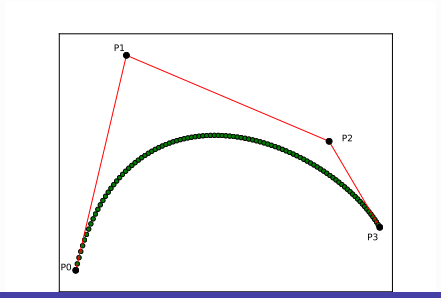
- Pasa por los extremos: $B(0) = P_0, \quad B(1) = P_n$
- Tangentes en los extremos: $B'(0) = n(P_1 - P_0), \quad B'(1) = n(P_n - P_{n-1})$

$$B'(t) = n \sum_{i=0}^{n-1} (P_{i+1} - P_i) B_i^{n-1}(t)$$

  Las tangentes en los extremos determinan las direcciones de los puntos de control internos.
- Invariante bajo transformaciones afines: para f afín,

$$f(B(t)) = \sum f(P_i) B_i^n(t), \qquad \{P_0, \ldots P_n\} \to \{f(P_0), \ldots f(P_n)\}$$

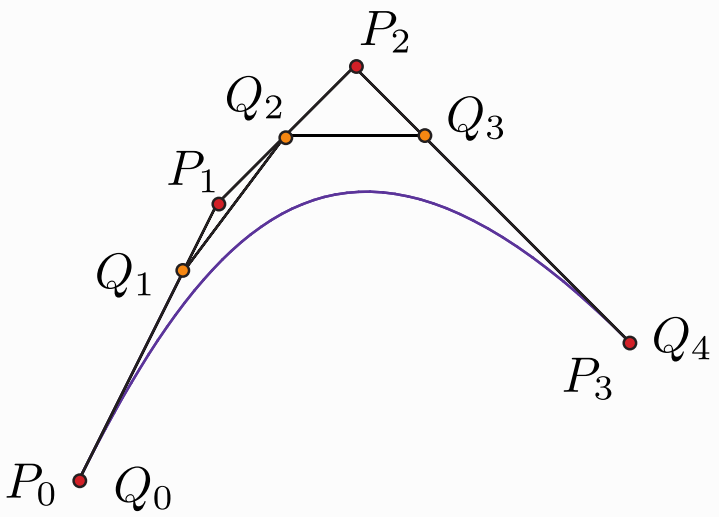
  Para transformar una curva de Bézier, es suficiente transformar los puntos de control.
- Envolvente convexa: la curva está contenida en el menor polígono convexo generado por los puntos de control $\{P_0, \ldots P_n\}$



Curvas de Bézier. Elevación del grado

- Una curva de Bézier de grado n (con puntos de control P_i) se puede expresar en términos de una curva de grado $n+1$, donde los nuevos puntos de control Q_i son:

$$Q_i = \frac{i}{n+1} P_{i-1} + \left(1 - \frac{1}{n+1}\right) P_i \qquad i = 0, \ldots, n+1 \qquad P_{-1} = P_{n+1} = 0$$



Ejemplo

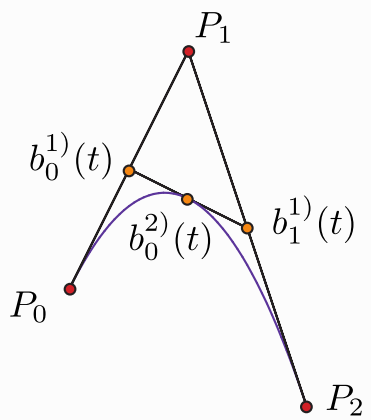
Curvas de Bézier. Algoritmo de De Casteljau

- La formulación con polinomios de Bernstein es equivalente al siguiente algoritmo de n iteraciones:

$$
\begin{aligned}
b_i^{1)}(t) &:= (1-t)P_i + tP_{i+1}, \quad i = 0, \dots, n-1 \\
b_i^{k)} &:= (1-t)b_i^{k-1)}(t) + tb_{i+1}^{k-1)}(t), \quad i = 0, \dots, n-k, \quad k = 1, \dots, n
\end{aligned}
$$

hasta llegar a un único punto: el valor $B(t) = b_0^{n)}(t)$

$$
\begin{array}{lcllcllcl}
b_0^{0)} = P_0 & \rightarrow & b_0^{1)}(t) & \rightarrow & b_0^{2)}(t) & \dots & \rightarrow & b_0^{n)}(t) \\
 & \nearrow & & \nearrow & & \dots & \nearrow & \\
b_1^{0)} = P_1 & \rightarrow & b_1^{1)}(t) & \rightarrow & b_1^{2)}(t) & & & \\
 & \nearrow & & \nearrow & & & & \\
b_2^{0)} = P_2 & \rightarrow & b_2^{1)}(t) & & & & & \\
\vdots & \nearrow & \vdots & & & & & \\
b_{n-1}^{0)} = P_{n-1} & \rightarrow & b_{n-1}^{1)}(t) & & & & & \\
 & \nearrow & & & & & & \\
b_n^{0)} = P_n & & & & & & &
\end{array}
$$

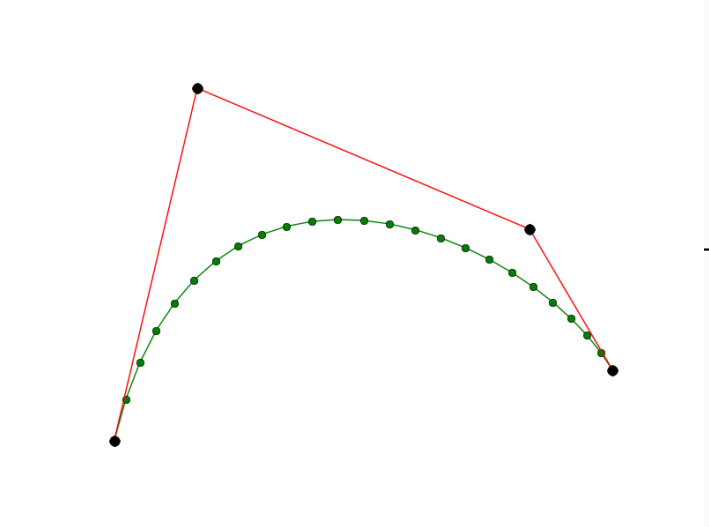


Ejemplo

- $B'(t) = n\left(b_1^{n-1)}(t) - b_0^{n-1)}(t)\right)$
- Fácil de programar. Rápido

Curvas de Bézier. Algoritmo de De Casteljau (cont.)

```
import matplotlib.pyplot as plt
import numpy as np

def B(coorArr, i, j, t):  #funcion recurvisa
    if j == 0:    return coorArr[i]
    return B(coorArr,i, j-1, t) * (1-t) + B(coorArr,i+1, j-1,t) * t

P=np.array([[0., 0.5],[0.2, 1.],[1., 0.8],[1.2,0.6]])
x=P[:,0];y=P[:,1];n=x.size
fig=plt.figure(1)

xb=[]; yb=[]
for t in np.linspace(0.,1.,25):
    a = B(x, 0, n - 1, t)
    b = B(y, 0, n - 1, t)
    xb.append(a); yb.append(b)

plt.plot(xb,yb,'g-o')
plt.plot(x,y,'r-',x,y,'ko',ms=8)
plt.show()
```

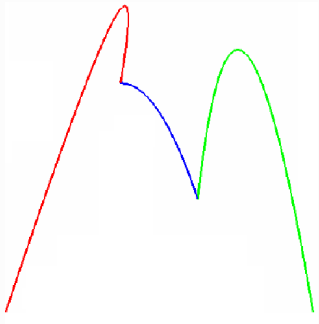


Curvas de Bézier a Trozos

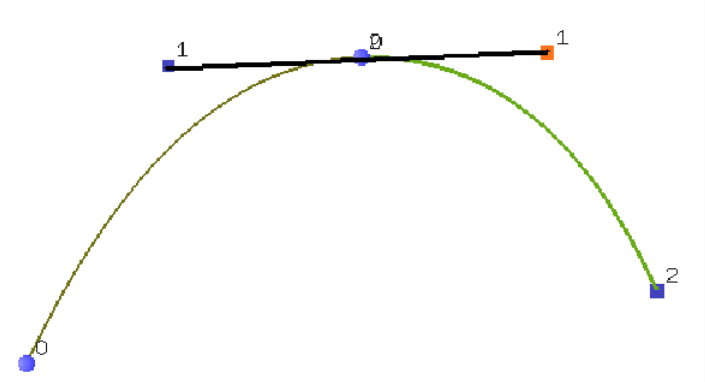
- Normalmente no se usan curvas de Bézier de grado > 3
- Para mejorar la aproximación, resulta más adecuado:
  - unir curvas de Bézier
  - utilizar B-splines
- Para generar figuras más complejas se pueden **unir varias curvas de Bézier** de grado pequeño, obteniendo curvas formadas por trozos
  - Es fundamental controlar cómo queremos que se unan nuestros trozos
    - Continuidad $\mathcal{C}^0$: el último punto de control de la curva S_i es igual al primer punto de control de la siguiente. (Hay picos) curva S_{i+1}
    - Continuidad $\mathcal{C}^1$: en este caso no sólo deben coincidir los puntos de los extremos, sino también sus derivadas laterales. (Genera uniones suaves)
    - Continuidad $\mathcal{G}^1$: en este caso no es estrictamente necesario que las derivadas coincidas, sino sólo que sean proporcioanles.
    - Continuidad $\mathcal{C}^2$: deben coincidir también las derivadas segundas. (A veces es deseable en 3D)

Curvas de Bézier a Trozos (cont.)

- **Continuidad $\mathcal{C}^0$:** (hay picos)

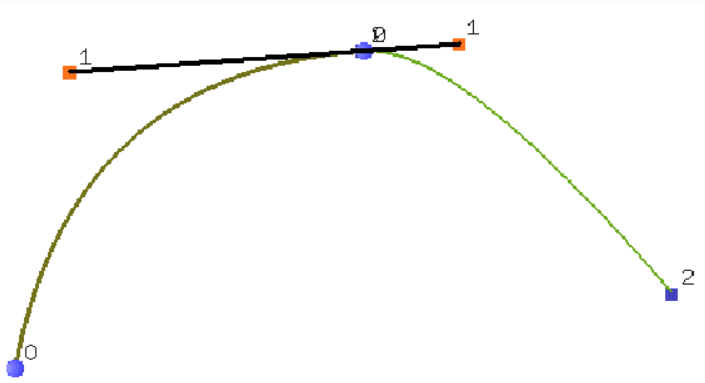


- **Continuidad $\mathcal{C}^1$:**
  - Gráficamente, además de coincidir el extremo de ambas curvas, P_i, este extremo y los dos puntos adyacentes, P_{i-1} y P_{i+1}, deben estar en la misma recta
  - Además se cumple: $Distancia(P_{i-1}, P_i) = Distancia(P_i, P_{i+1})$



Curvas de Bézier a Trozos (cont.)

- **Continuidad $\mathcal{G}^1$:**
  - Gráficamente, además de coincidir el extremo de ambas curvas, P_i, este extremo y los dos puntos adyacentes, P_{i-1} y P_{i+1}, deben estar en la misma recta (igual que en $\mathcal{C}^1$)
  - No es necesario que las distancias sean iguales



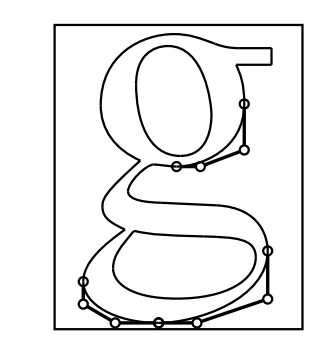
Curvas de Bézier a Trozos (cont.)

Propiedades:

- Control local: al mover un punto modificamos sólo una curva (o a lo sumo dos, si se trata de un punto de unión); esto no ocurriría en Bézier simple
- Se pueden combinar curvas y rectas en una única figura contínua
- Podemos modelar figuras muy complejas

Aplicación:

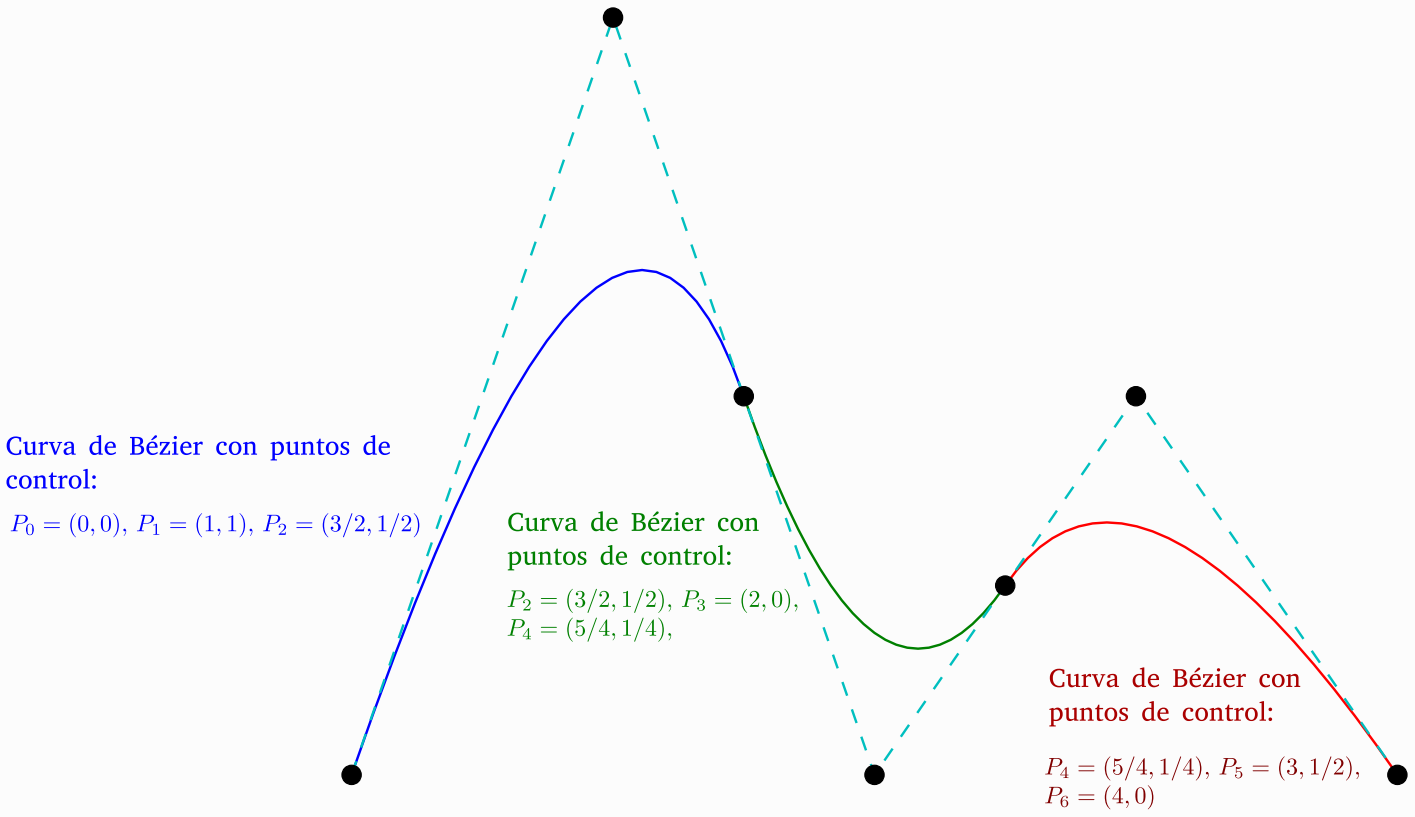
- fuentes PostScript, definidas pegando curvas de Bézier de grado 3.



Curvas de Bézier a Trozos (cont.)

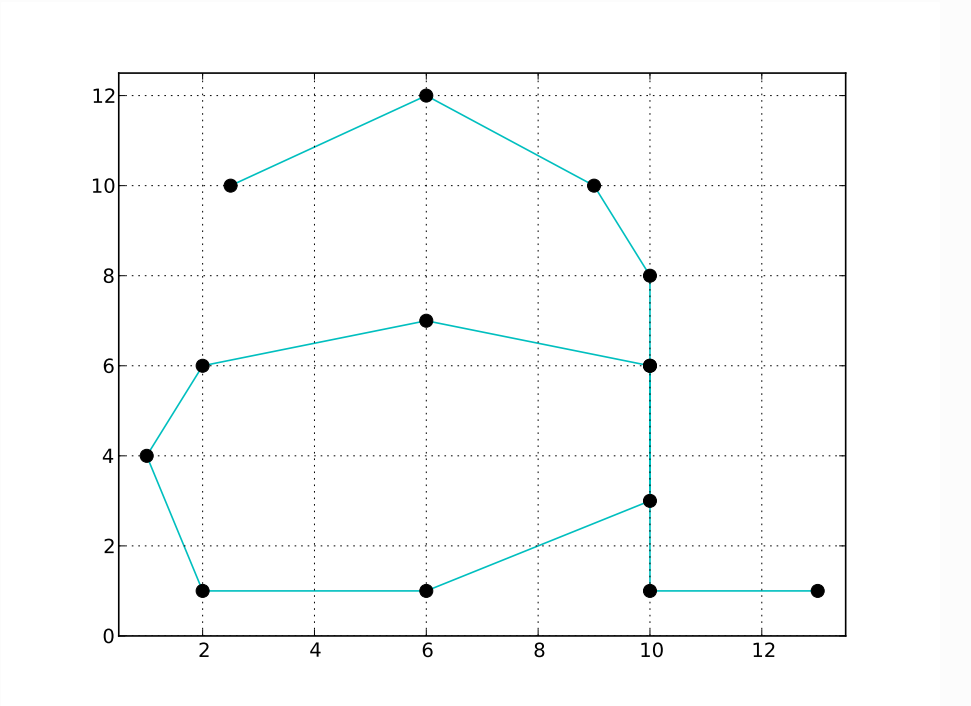
```
import matplotlib.pyplot as plt
import numpy as np

def B(coorArr, i, j, t):
    if j == 0:
        return coorArr[i]
    return B(coorArr, i, j - 1, t) * (1 - t) + B(coorArr, i + 1, j - 1, t)
          * t

P=np.array([[0., 0.],[1., 1.],[3./2., 1./2.],
             [2.0,0.],[5./2.,1./4.],[3.,1./2.],[4.,0.]])
fig=plt.figure(1)
ini=0; fin=3
for k in range(0,3):
    x=P[ini:fin,0];  y=P[ini:fin,1]; n=x.size
    xb=[];  yb=[]
    for t in np.linspace(0.,1.,25):
        a = B(x, 0, n - 1, t)
        b = B(y, 0, n - 1, t)
        xb.append(a)
        yb.append(b)
    plt.plot(xb,yb)
    ini=fin-1
    fin=ini+n

plt.plot(P[:,0],P[:,1],'c--',P[:,0],P[:,1],'ko',ms=8)
plt.show()
```

Curvas de Bézier a Trozos (cont.)



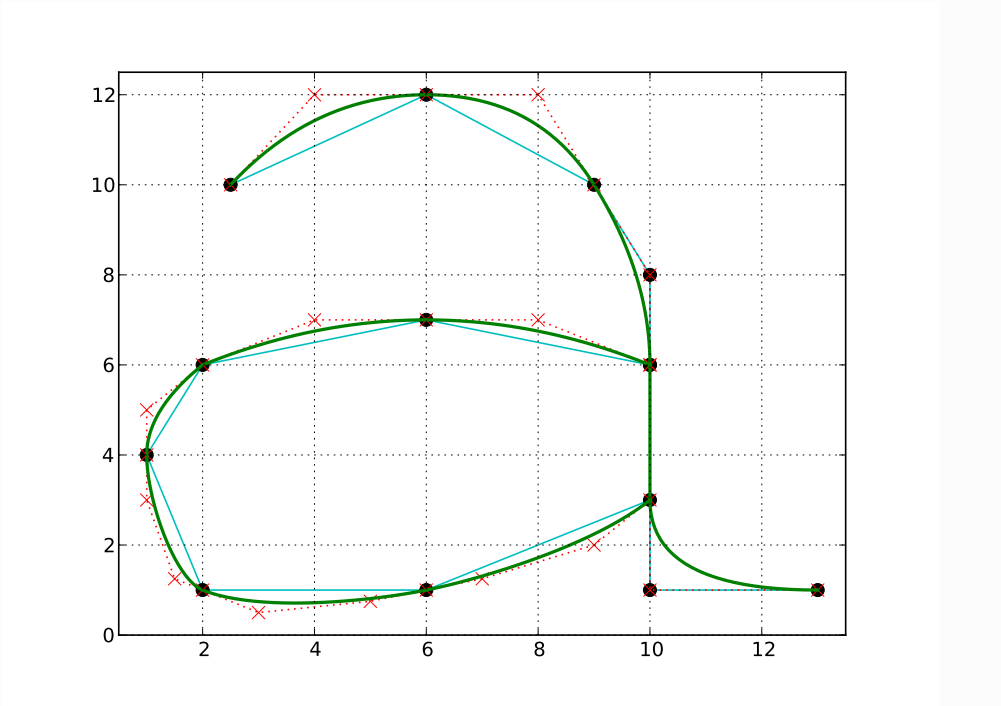
- El proceso de ir pegando trozos de curvas puede resultar tedioso

Curvas de Bézier a Trozos. Ejercicio



$$x = [2{,}5, 6., 9., 10., 10., 6., 2., 1., 2, 6., 10., 10., 10., 13]$$
$$y = [10., 12., 10., 8., 6., 7., 6., 4., 1., 1., 3., 6., 1., 1.]$$

Curvas de Bézier a Trozos. Ejercicio



Curvas de Bézier Racionales

- Las cónicas (círculos, elipses, hipérbolas, etc.) no se pueden representar usando curvas de Bézier (polinomios).
- Coordenadas homogéneas: en $\mathbb{R}^2$ los puntos se representan por 3 coordenadas
  - $P = (x, y, 1)$ ó, equivalentemente, $P = (\omega x, \omega y, \omega)$, $\omega \neq 0$
  - ω se llama peso del punto P.
  - Para recuperar P en $\mathbb{R}^2$, dividimos sus dos primeras coordenadas por ω
  - Los puntos con peso 0 son "puntos del infinito"

  De forma análoga en $\mathbb{R}^2$:
- Dados los puntos $P_0, \ldots, P_n$, considerar los puntos en coordenadas homogéneas: $P_0^\omega = (\omega_0 P_0, \omega_0), \ldots, P_n^\omega = (\omega_n P_n, \omega_n)$, con $\omega_i \neq 0$ (pesos), $i = 0, \ldots, n$

Curvas de Bézier Racionales

- Contruimos la curva de Bézier que pasa por estos puntos:

$$B^{\omega}(t) = \sum_{i=0}^{n} P_i^{\omega} B_i^n(t) = \left(\sum_{i=0}^{n} P_i \omega_i B_i^n(t), \sum_{i=0}^{n} \omega_i B_i(t) \right)$$

- Recuperamos la curva en coordenadas no homogéneas. **Curva de Bézier racional**

$$R(t) = \sum_{i=0}^{n} \frac{\omega_i B_i^n(t)}{\sum_{k=0}^{n} \omega_k B_k^n(t)} P_i = \frac{\sum_{i=0}^{n} P_i \omega_i B_i^n(t)}{\sum_{k=0}^{n} \omega_k B_k^n(t)}$$

- Si $\omega_i = 1$, se tiene la curva de Bézier, puesto que $\sum_{k=0}^{n} B_k^n(t) = 1$

Curvas de Bézier Racionales

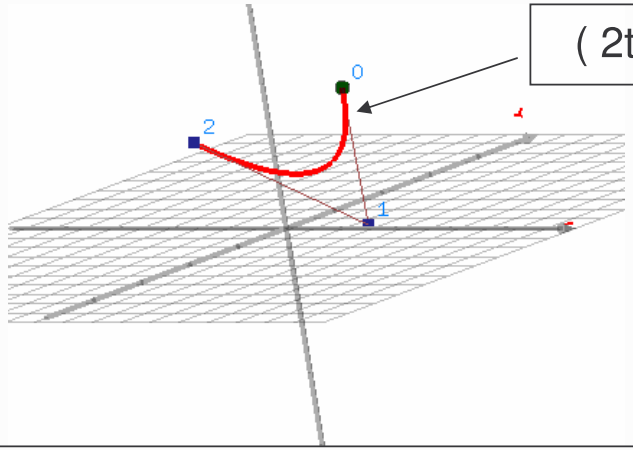
- Las funciones de base, $R_i(t)$ suma 1

$$R_i(t) = \frac{\omega_i B_i^n(t)}{\sum_{k=0}^{n} \omega_k B_k^n(t)}$$

- La curva es una combinación convexa de los puntos de control
- Interpolación en los extremos y tangencia al polinomio de control en los extremos
- La última coordenada (en homogéneas) de cada punto, ω_i, se puede utilizar para dar más o menos peso a un punto. Si se eleva su peso, la curva se dobla hacia el punto
- Si utilizamos puntos con peso cero, podremos construir secciones cónicas (son curvas de Bézier racionales de grado 2). Se pueden ver como proyecciones sobre el plano $z = 1$ de parábolas en 3D
- Las curvas de Bézier racionales son útiles para representar cónicas, herramienta utilizada en la industria aerodinámica

Curvas de Bézier Racionales

Ejemplo: curva de Bézier con puntos de control $P_0 = (0,1,1)$, $P_1 = (1,1,0)$, $P_2 = (0,-1,1)$

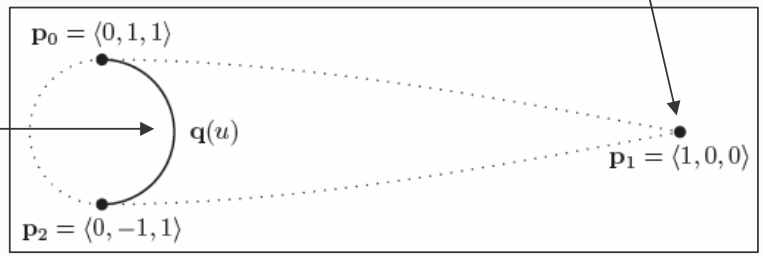


$$\left(2t(1-t)\ ,\ (1-t)^2-t^2\ ,\ (1-t)^2+t^2 \right)$$

proyección en z=1

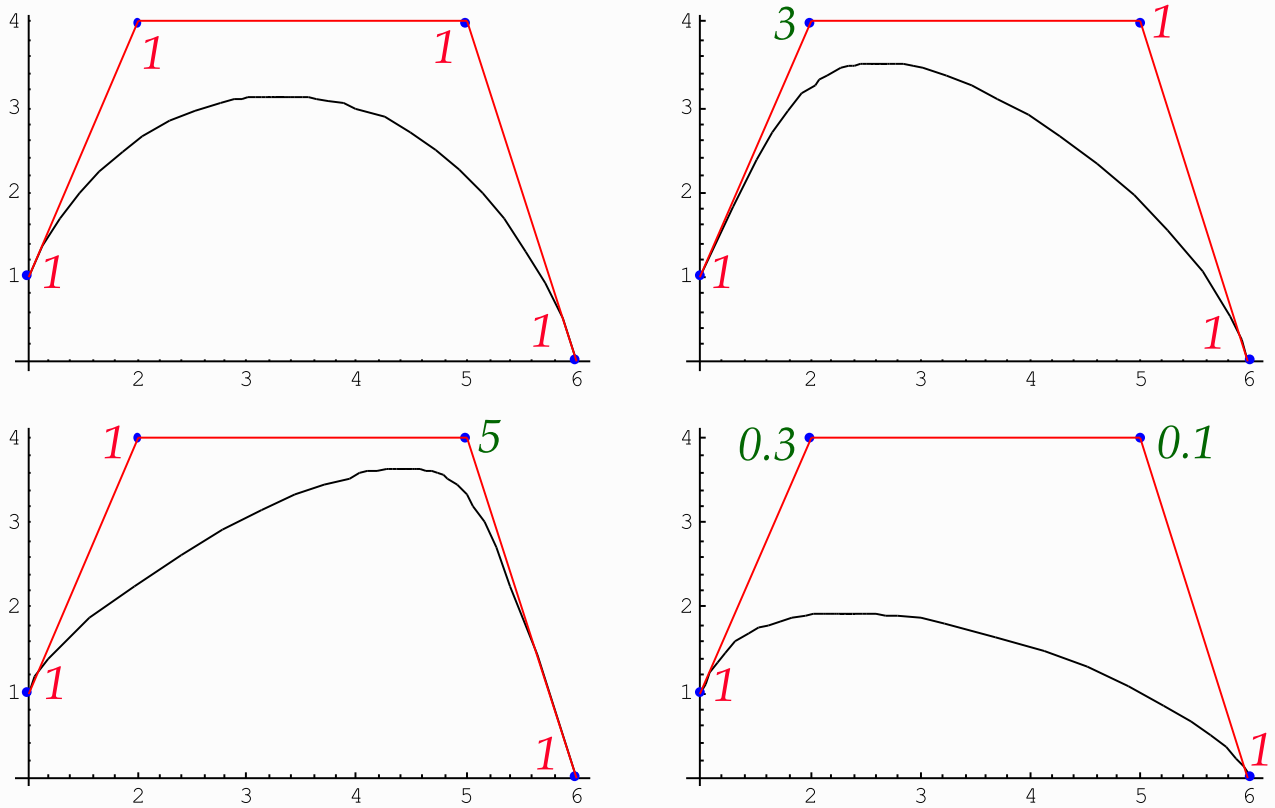
curva de Bezier racional:

$$\left(\frac{2t(1-t)}{(1-t)^2+t^2}\ ,\ \frac{(1-t)^2-t^2}{(1-t)^2+t^2} \right)$$



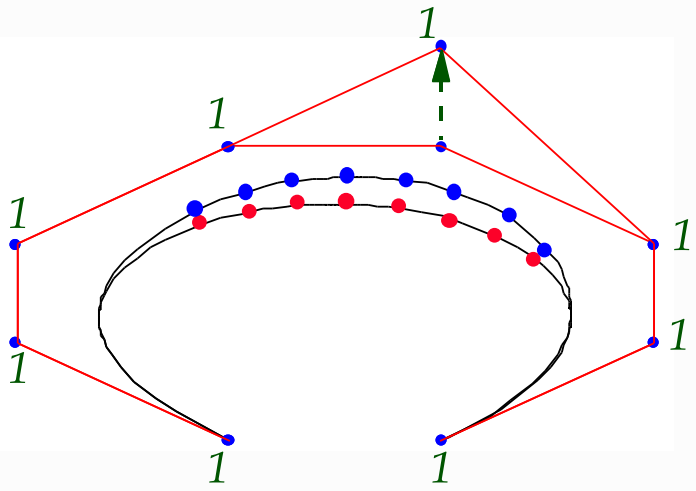
Curvas de Bézier Racionales

- Modificando los pesos:
  - $\omega_i > 1$: la curva se aproxima a P_i
  - $\omega_i < 1$: la curva se aleja P_i

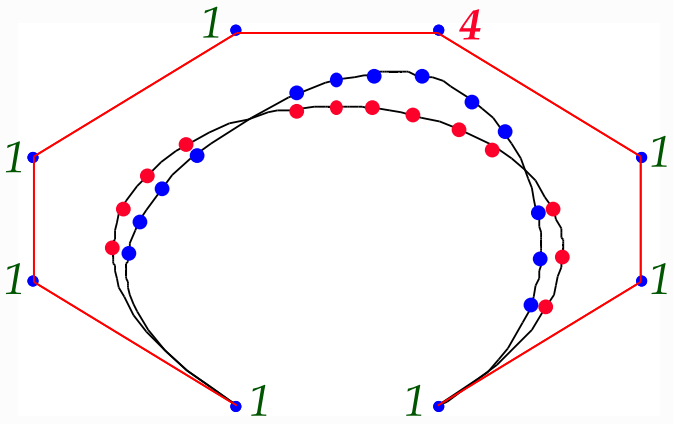


Curvas de Bézier Racionales

- Influencia de los pesos: el efecto de cambiar es diferente a mover los puntos de control



Se modifica un punto de control



Se modifica un peso

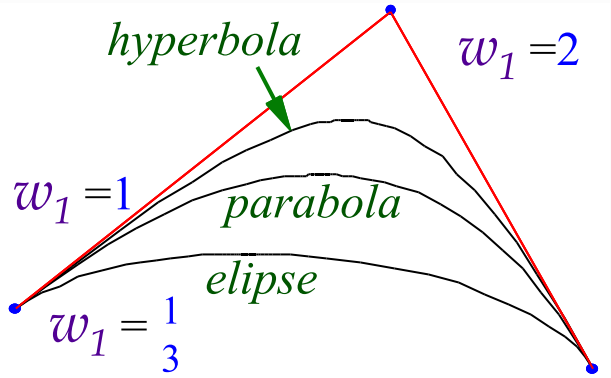
Curvas de Bézier Racionales

- Sea $c(t)$ un punto de un cónica. Entonces, existen números ω_0, ω_1 y ω_2 y P_0, P_1 y P_2 puntos bidimensionales tales que:

$$c(t) = \frac{\omega_0 P_0 B_{02}(t) + \omega_1 P_1 B_{12}(t) + \omega_2 P_2 B_{22}(t)}{\omega_0 B_{02}(t) + \omega_1 B_{12}(t) + \omega_2 B_{22}(t)}$$

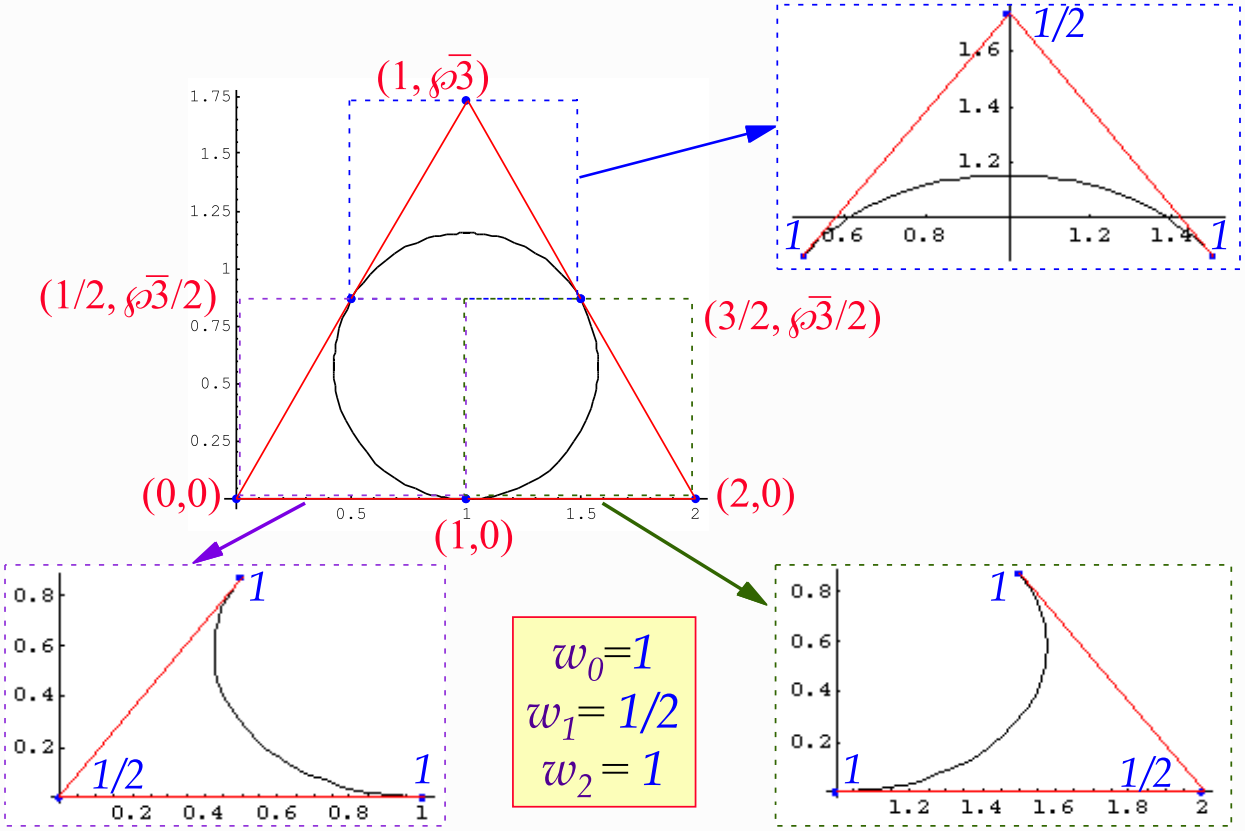
Si consideramos $\omega_0 = \omega_2 = 1$ y se define $s = \frac{\omega_1}{1+\omega_1}$

  - $s = 1/2$ se tiene un arco parabólico
  - $s < 1/2$ se tiene un arco elíptico
  - $s > 1/2$ se tiene un arco hiperbólico



Curvas de Bézier Racionales

- Ejemplo: círculo



B-Spline

- Las B-Splines generalizan las curvas de Bézier a trozos
- La "B" es por base o blending, no por Bézier
- Son los más utilizados en la práctica:
  - B-splines cuadráticos: fuentes True Type
  - B-splines cúbicos: los más comunes en programas de diseño gráfico, como por ej. Photoshop
- En general, no pasan por ningún punto de control (ni siquiera los extremos), aunque se puede formar a que lo hagan
- Caso más general que Bézier.
  Las curvas de Bézier son un tipo de B-Splines
- Principales ventajas sobre las curvas de Bézier:
  - El grado de la curva se escoge a priori. Por tanto, el grado es de grado acotado (aún definido para n puntos)
  - Tiene carácter local: cada vértice de control va asociado a una función base, que tiene soporte local

B-Spline

- Datos para construir una curva B-Spline:
  - orden de la curva k (la curva está formada por polinomios a trozos de grado k)
  - $n+1$ vértices o puntos de control (llamados puntos de Boor) $P_0, P_1, \ldots, P_n$ (puntos del plano o del espacio)
  - vector de nodos (knots) $T = [t_0, t_1, \ldots, t_{n+k+1}]$, tales que $t_i \in \mathbb{R}$, $t_i \leq t_{i+1}$, $k < n$
- Nota:
  - Los nodos se pueden repetir
  - los vértices se pueden repetir

B-Spline. Construcción

- Dados el orden k, $n+1$ puntos de control $P_0, \ldots, P_n$, y el vector de nodos $T = [t_0, t_1, \ldots, t_{n+k+1}]$, se define la curva B-spline $X(t)$ de orden k asociada a estos datos como

$$X(t) = \sum_{i=0}^{n} P_i N_{ik}(t) \quad \text{con } t \in [t_k, t_{n+1}]$$

  siendo N_{ik} las funciones base B-spline de grado $k-1$ asociadas al vector de nodos T

- Fórmula de **Cox-De Boor** (cálculo de las funciones base):

$$N_{i1}(t) = \begin{cases} 1 \text{ si } t_i \leq t \leq t_{i+1} \\ 0 \text{ en otro caso} \end{cases} \qquad k = 1$$

$$N_{ik}(t) = \frac{t - t_i}{t_{i+k-1} - t_i} N_{i,k-1}(t) + \frac{t_{i+k} - t}{t_{i+k} - t_{i+1}} N_{i+1,k-1}(t) \qquad k \geq 1$$

Si algún denominador es 0, $N_{ik}(t) = 0$

B-Spline. Construcción (cont.)

Relaciones de dependencia de las funciones de base: Cálculo de $N_{i,k}(t)$

$$
\begin{array}{ccccccccc}
N_{i,k}(t) & & & & & & & & \\
\downarrow & \searrow & & & & & & & \\
N_{i,k-1}(t) & & N_{i+1,k-1}(t) & & & & & & \\
\downarrow & \searrow & \downarrow & \searrow & & & & & \\
N_{i,k-2}(t) & & N_{i+1,k-2}(t) & & N_{i+2,k-2}(t) & & & & \\
\downarrow & \searrow & \downarrow & \searrow & \downarrow & \searrow & & & \\
\vdots & & \vdots & & \dots & & & & \\
N_{i,0}(t) & & N_{i+1,0}(t) & & N_{i+2,0}(t) & \dots & \dots & & N_{i+k,0}(t)
\end{array}
$$

Propiedades de las funciones base:

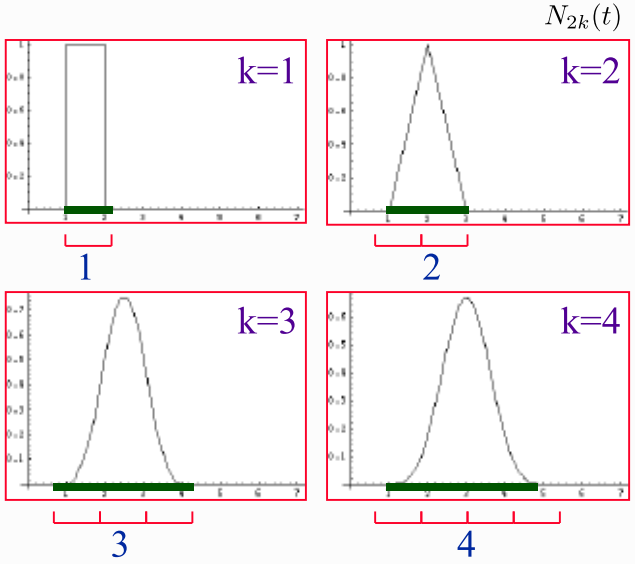
- Las funciones de base de orden k suman 1: $\sum_{i=0}^{n} N_{ik}(t) = 1$ cuando $t \in [t_k, t_{n+1}]$
- $N_{ik}(t) > 0 \Leftrightarrow t \in (xi, x_{i+k+1})$ y en el resto valen 0
- Para $k > 0$, $N_{ik}(t)$ tiene un único máximo
- El dominio de $N_{i,k}(t)$ es (t_i, t_{i+k+1}), porque la función $N_{i,k}(t)$ depende de $N_{i,0}(t)$, $N_{i+1,0}(t), \ldots, N_{i+k,0}(t)$

B-Spline. Vectores de nodos

- Construcción de los **vectores de nodos**:
  - vectores de nodos uniformes o equiespaciados
  - vectores de nodos "clamped" o agarrados
  - vectores de nodos no uniformes

B-Spline. Vectores de nodos

- **Vectores de nodos uniformes**:
  Los puntos t_i aparecen equiespaciados. En la práctica se suelen escribir de uno de los siguientes modos:
  - Empezando en cero, $t_0 = 0$, y de modo que $t_i = t_{i-1} + 1$. Ej: $[0, 1, 2, 3, 4]$
  - De forma que $t_0 = 0$, $t_{n+k+1} = 1$ y los demás están repartidos en intervalos iguales entre 0 y 1. Ej: [0; 0,25; 0,5; 0,75; 1]
    - Las funciones de base son copias trasladadas de la función $N_{0,k}(t)$:
      $$N_{i,k}(t) = N_{0,k}(t - i), \text{ para } i > 0$$
      Esto supone una ventaja en el cómputo de las funciones base
    - La influencia de cada función de base, $N_{i,k}$, se limita a k intervalos.

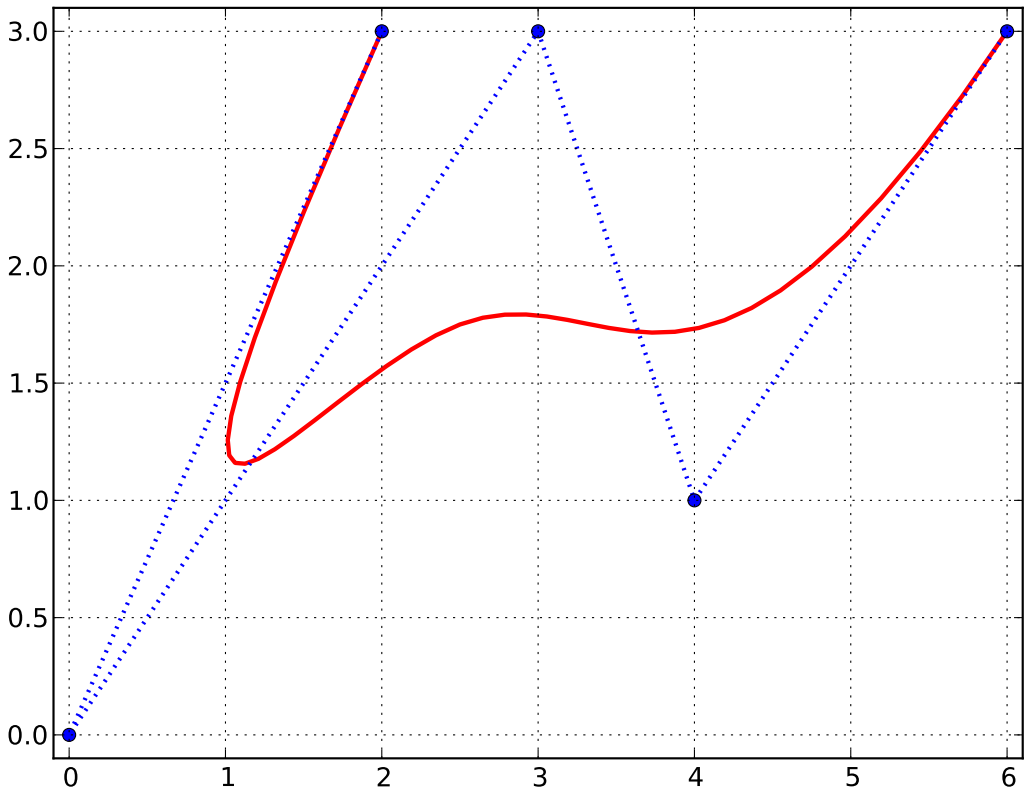


B-Spline. Vectores de nodos

- **Vectores de nodos "clamped" o agarrados**:
  - Los nodos del inicio y del final del vector de nodos tienen multiplicidad $k+1$, y los nodos interiores están equiespaciados. Si el vector de nodos es $[t_0, \ldots, t_{n+k+1}]$ se suele considerar:
    - $t_i = 0$ para $0 \leq i \leq k$
    - $t_i = i - k$ para $k+1 \leq i \leq n$
    - $t_i = n - k + 1$ para $n+1 \leq i \leq n+k+1$

    Ejemplos:
    - $k = 1: \ [0, 0, 1, 2, 2]$
    - $k = 2: \ [0, 0, 0, 1, 1, 1]$
    - $k = 3: \ [0, 0, 0, 0, 1, 2, 3, 3, 3, 3]$

B-Spline



B-Spline

Propiedades de las curvas B-Spline:

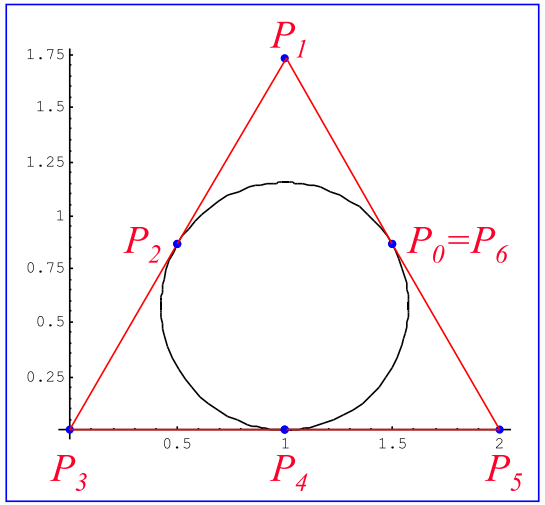
- Control local: como cada $N_{ik}(t)$ tiene soporte local, al modificar un vértice de control tan sólo se modifica una porción de la curva
- Invarianza afín: la curva es una conbinación convexa, de sus vértices de control, así que respeta transformaciones afines

Cuvas B-Spline Racionales

- Introduciendo pesos ω_i se tiene la curva B-spline racional

$$R(t) = \sum_{i=0}^{n} \frac{\omega_i N_{ik}(t)}{\sum_{j=0}^{n} \omega_j N_{jk}}(t) P_i = \frac{\sum_{i=0}^{n} P_i \omega_i N_{ik}(t)}{\sum_{i=0}^{n} \omega_i N_{ik}(t)}$$

- Ejemplo: Se puede obtener un círculo usando siete puntos de control $\{P_0, P_1, P_3, P_4, P_5, P_6\}$ y como vector de nodos $T = [0, 0, 0, 1/3, 1/3, 2/3, 1, 1, 1]$ y pesos $\omega = \{1, 1/2, 1, 1/2, 1, 1/2, 1\}$



Non-Uniform Rational B-splines (NURBS)

- Caso más general: las curvas de Bézier y B-spline son casos particulares de NURBS
- Non-Uniform Rational B-splines (NURBS): B-spline racional con nodos no uniformes
- Hoy en día, NURBS es una de las entidades geométrica más importantes en diseño
- Incorporada en los programas más populares de CAD/CAM y Diseño Gráfico

Superficies de Bézier

- Dado el conjunto de puntos $P_{ij} \in \mathbb{R}^3$, $i = 0, 1, \ldots, m$, $j = 0, 1, \ldots, n$

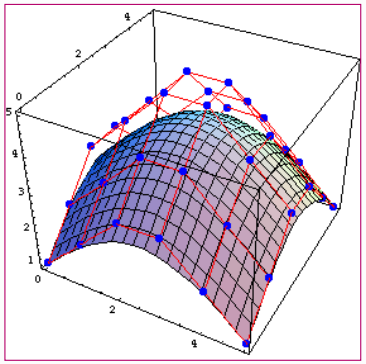
$$P = \{ \begin{array}{l} \{P_{00}, P_{01}, \ldots, P_{0n}\}, \\ \{P_{10}, P_{11}, \ldots, P_{1n}\}, \\ \vdots \\ \{P_{m0}, P_{m1}, \ldots, P_{mn}\}\} \end{array}$$

- La superficie de Bézier asociada al conjunto de puntos P se define como

$$S(u,v) = \sum_{i=0}^{m} \sum_{j=0}^{n} P_{ij} B_i^m(u) B_j^n(v)$$

donde $B_i^m(u)$ y $B_j^n(v)$ representan los polinomios de Bernstein de grados m y n en las variables u y v (u, $v \in [0,1]$), respectivamente

x	y	z	x	y	z	x	y	z	x	y	z	x	y	z	x	y	z
0	0	1	0	1	2	0	2	3	0	3	3	0	4	2	0	5	1
1	0	2	1	1	3	1	2	4	1	3	4	1	4	3	1	5	2
2	0	3	2	1	4	2	2	5	2	3	5	2	4	4	2	5	3
3	0	3	3	1	4	3	2	5	3	3	5	3	4	4	3	5	3
4	0	2	4	1	3	4	2	4	4	3	4	4	4	3	4	5	2
5	0	1	5	1	2	5	2	3	5	3	3	5	4	2	5	5	1

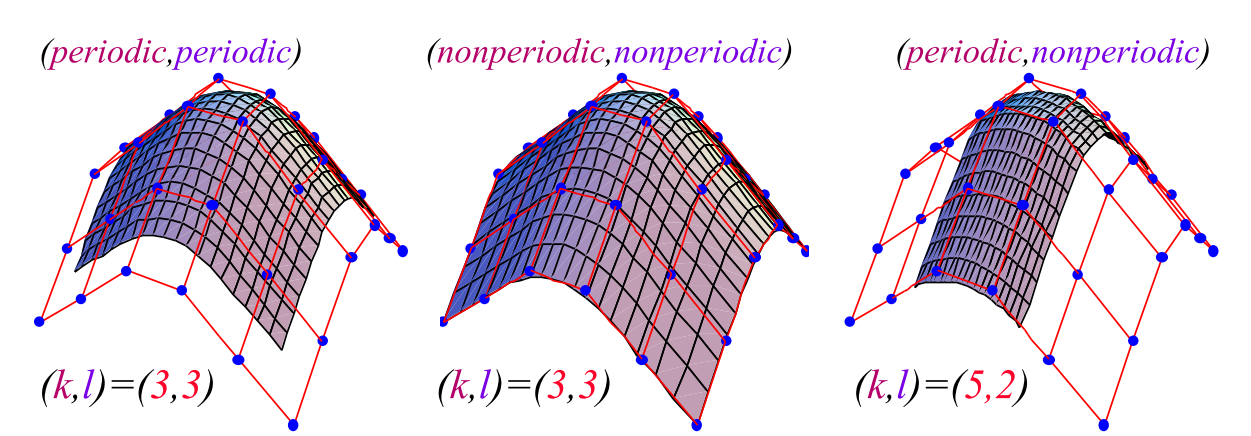


Superficies B-splines

- Una superficie B-spline de orden k en la dirección u y orden l en la dirección v se define como

$$S(u,v) = \sum_{j=0}^{m} \sum_{i=0}^{n} P_{ij} N_{ik}(u) N_{jl}(v)$$

donde P_{ij} son los puntos de control y $N_{ik}(v)$ y $N_{jl}(v)$ son las funciones base B-splines definidas en los vectores de nodos
$U = [u_0, u_1, \ldots, u_{p-1}, u_p]$ y $U = [v_0, v_1, \ldots, v_{q-1}, v_q]$ $(p = m + k,\ q = l + n)$



Ejemplo

- Movimiento vehículo usando spline:
  `http://www.ambiera.at/copperlicht/demos/demo_toycar_external.html`
- Animación del tornado:
  `http://www.webgl.com/2012/04/webgl-demo-tornado/`
- Modelos basados en splines:
  `http://www.ibiblio.org/e-notes/webgl/models.htm#spline`
- Curvas Nurbs:
  `http://miaumiau.cat/examples/nurbs/nurbsCurves.swf`
- Superficies Nurbs:
  `http://miaumiau.cat/examples/nurbs/nurbsSurfaces.swf`